

EE4410 Cyber Security of Power Grids

Assignment 2: Cryptography Applications

Name: Zubair Al-Zubi

Student Number: 5342007

Date of Submission: May 28, 2025

Introduction

Objectives:

- Understand the application of symmetric and asymmetric cryptography.
- Test and demonstrate symmetric encryption and decryption using AES.
- Test and demonstrate asymmetric encryption and decryption using RSA.
- Test and demonstrate different digital signatures algorithms (i.e., RSASSA-PSS and ECDSA)

In this laboratory you will be communicating with another student using an unsecure communication medium. To simplify the exchange of messages between yourselves, JSONbin.io website and its public API will be used. Keep in mind that this is for real an unsecure communication channel given that ANY message published on the website can be read by ANYONE on the public internet!

Steps to use the JSONbin.io API:

- Create an account and sign in to JSONbin.io.
- From your JSONbin.io dashboard, under the "API KEYS" tab, you can obtain your personal X-Master-Key to be used to access the API. This key is unique to each JSONbin.io account and should be kept private!
- From your JSONbin.io dashboard, under the "BINS" tab, you can create a new PUBLIC bin. You can insert the following JSON to successfully initialize it: `{"Lab": "Crypto"}`. Note: remember to set the bin as public.
- Save your BIN ID. This will be your "unsecure upload communication channel". Your published message will be saved in this BIN.
- Ask your friend to provide their BIN ID. This will be the "unsecure download communication channel" to be used for receiving the messages from your friend.

Utility functions:

In the "Utility functions" chapter, you can find all the functions to be used to upload/download messages to JSONbin, encrypt/decrypt messages using AES/RSA, generate RSA/EC keys, and generate/validate digital signatures using RSASSA-PSS and ECDSA. Also, examples on how to use these functions are provided. The code in the "Utility functions" chapter is not meant to be modified; it is there to help you complete the tasks in the "Assignment" chapter.

JSONbin step-by-step tutorial

The screenshot shows the JSONbin.io dashboard. The sidebar on the left contains navigation links: DASHBOARD, BINS (1), COLLECTIONS, SCHEMA DOCS, API KEYS, MY ACCOUNT, and API REFERENCE. The main content area is titled 'Bins' and contains a table of existing bins. A 'Create a Bin' modal is open, showing a form to create a new bin. The modal has a 'Create A Bin' button (2), a 'Save Bin' button (5), and a text area for the bin content (4) containing the JSON object `{"Lab": "Crypto"}`. An important warning message is displayed in the modal: 'Important: Setting the Bin permission to Public will allow anyone to READ & UPDATE the Bin. Anyone Updating the Public Bin without X-Master-Key will not be able to Disable Version Control while Updating the Bin.' The modal also has a 'Create A Bin' button (3) and a 'Save Bin' button (5).

Bin ID	Name	Created
67dfce8f8561e97a50f12f47	Untitled	09:04 23-03-2025 (UTC)
67dd83378a456b79667a17e7	Untitled	15:18 21-03-2025 (UTC)
67dd82d08561e97a50f046c9	Untitled	15:16 21-03-2025 (UTC)
67dd52d48960c979a575c1dc	Untitled	11:51 21-03-2025 (UTC)
67dd51b88960c979a575c155	Untitled	11:47 21-03-2025 (UTC)

✓ Define and import environment variables

```
1 from google.colab import userdata
2
3 JSONbin_X_Master_Key = userdata.get('JSONbin_X_Master_Key')    # Your personal API key to needed to use
4
5 JSONbin_Bin_ID_A = '6836102f8561e97a501c4973'                # Your own BIN. Here is where your messages will be
6 JSONbin_Bin_ID_B = '68373cd58561e97a501caf9c'                # Your friend BIN. Here is from where your friend n
```

✓ Import required modules

```
1 !pip install pycryptodome
2
3 import base64
4 import requests
5 import json
6 import binascii
7
8 from Crypto.Cipher import AES, PKCS1_OAEP
9 from Crypto.Hash import SHA256
10 from Crypto.PublicKey import RSA, ECC
11 from Crypto.Signature import DSS, pss
12 from Crypto import Random
```

 Collecting pycryptodome
 Downloading pycryptodome-3.23.0-cp37-abi3-manylinux_2_17_x86_64.manylinux2014_x86_64.whl.metadata (3.4 kB)
 Downloading pycryptodome-3.23.0-cp37-abi3-manylinux_2_17_x86_64.manylinux2014_x86_64.whl (2.3 MB)
 2.3/2.3 MB 27.3 MB/s eta 0:00:00
Installing collected packages: pycryptodome
Successfully installed pycryptodome-3.23.0

✓ Utility functions

✓ JSONbin

Available functions:

- `send_message_to_jsonbin(bin_id, message, secret_key) → response`
- `receive_message_from_jsonbin(bin_id, secret_key) → response`

```
1 # Introduction
2 # This code demonstrates how to send and receive messages through JSONbin.
3 # JSONbin is a service that allows you to store and retrieve JSON data easily.
4
5 # JSONbin Utility functions
6 def send_message_to_jsonbin(bin_id, message, secret_key):
7     """
8     Sends a message to a JSONbin.
9
10    Args:
11        bin_id (str): The ID of the JSONbin.
12        message (dict): The message to send.
13        secret_key (str): Personal JSONbin API Master-KEY.
14
15    Returns:
16        dict: The response from JSONbin.
17    """
18    url = f"https://api.jsonbin.io/v3/b/{bin_id}"
19    headers = {
20        "Content-Type": "application/json",
21        "X-Master-Key": secret_key,
22    }
23    response = requests.put(url, headers=headers, json=message)
24    return response.json()
25
26
27 def receive_message_from_jsonbin(bin_id, secret_key):
28     """
```

```

29     Receives a message from a JSONbin.
30
31     Args:
32         bin_id (str): The ID of the JSONbin.
33         secret_key (str): Personal JSONbin API Master-KEY.
34
35     Returns:
36         dict: The message received from JSONbin.
37     """
38     url = f"https://api.jsonbin.io/v3/b/{bin_id}/latest"
39
40     headers = {
41         "X-Master-Key": secret_key,
42     }
43     response = requests.get(url, headers=headers)
44     if response.status_code == 200:
45         return response.json()["record"]
46     else:
47         return None

```

✓ Example Usage

```

1 # Replace with your JSONbin ID and secret key
2 bin_id = JSONbin_Bin_ID_A          # JSONbin where to send/receive the message
3 master_key = JSONbin_X_Master_Key  # JSONbin X-Master-Key
4
5 # Send a message
6 message = {"message": "Its lights out and away we go", "signature": "79672f07e38abbb8f6369453e016aacd"}
7 send_response = send_message_to_jsonbin(bin_id, message, master_key)
8 print(f"Send response: {send_response}")
9
10 # Receive the latest message
11 received_message = receive_message_from_jsonbin(bin_id, master_key)
12 print(f"Received message: {received_message}")
13

```

 Send response: {'record': {'message': 'Its lights out and away we go', 'signature': '79672f07e38abbb8f6369453e016aacd'}},
 Received message: {'message': 'Its lights out and away we go', 'signature': '79672f07e38abbb8f6369453e016aacd'}

✓ Advanced Encryption Standard (AES)

Available functions:

- `encrypt(message, passphrase, key_length, mode) → (nonce/iv +) encrypted_message (+ tag)`
- `decrypt(ciphertext, passphrase, key_length, mode) → message`

Supported modes of operation:

- Electronic Codebook Mode (ECB)
- Cipher Block Chaining Mode (CBC)
- Cipher Feedback Mode (CFB)
- Output Feedback Mode (OFB)
- Counter Mode (CTR)
- Counter with CBC-MAC mode (CCM)
- Galois/Counter Mode (GCM)
- Offset Codebook Mode (OCB)

Supported key lengths:

- 128
- 192
- 256

```

1 # AES docs: https://pycryptodome.readthedocs.io/en/latest/src/cipher/aes.html
2
3 import base64
4 from Crypto.Cipher import AES
5 from Crypto.Util.Padding import pad, unpad
6 from Crypto.Hash import SHA256
7 from Crypto.Random import get_random_bytes

```

```

8
9 def encrypt(message, passphrase, key_length=256, mode=AES.MODE_CBC):
10     """Encrypts a message using AES."""
11     if key_length not in [128, 192, 256]:
12         raise ValueError("Invalid key length. Must be 128, 192, or 256.")
13
14     # Generate AES key from passphrase
15     key = SHA256.new(passphrase.encode()).digest()[:key_length // 8]
16
17     if mode in [AES.MODE_CBC, AES.MODE_CFB, AES.MODE_OFB]:
18         iv = get_random_bytes(AES.block_size) # 16-byte IV
19         cipher = AES.new(key, mode, iv)
20         if mode in [AES.MODE_CFB, AES.MODE_OFB]:
21             encrypted_message = cipher.encrypt(message.encode())
22         else:
23             encrypted_message = cipher.encrypt(pad(message.encode(), AES.block_size))
24         return base64.b64encode(iv + encrypted_message).decode('utf-8')
25
26     elif mode in [AES.MODE_CCM, AES.MODE_GCM, AES.MODE_OCB, AES.MODE_CTR]:
27         nonce = get_random_bytes(12) # 12-byte nonce
28         cipher = AES.new(key, mode, nonce=nonce)
29         if mode in [AES.MODE_CCM, AES.MODE_GCM, AES.MODE_OCB]:
30             encrypted_message, tag = cipher.encrypt_and_digest(message.encode())
31             return base64.b64encode(nonce + encrypted_message + tag).decode('utf-8')
32         else:
33             encrypted_message = cipher.encrypt(message.encode())
34             return base64.b64encode(nonce + encrypted_message).decode('utf-8')
35
36     else:
37         cipher = AES.new(key, mode)
38         if mode in [AES.MODE_ECB]:
39             encrypted_message = cipher.encrypt(pad(message.encode(), AES.block_size))
40         else:
41             encrypted_message = cipher.encrypt(message.encode())
42         return base64.b64encode(encrypted_message).decode('utf-8')
43
44 def decrypt(encrypted_message, passphrase, key_length=256, mode=AES.MODE_CBC):
45     """Decrypts a message using AES."""
46     if key_length not in [128, 192, 256]:
47         raise ValueError("Invalid key length. Must be 128, 192, or 256.")
48
49     # Generate AES key from passphrase
50     key = SHA256.new(passphrase.encode()).digest()[:key_length // 8]
51
52     encrypted_message = base64.b64decode(encrypted_message)
53
54     if mode in [AES.MODE_CBC, AES.MODE_CFB, AES.MODE_OFB]:
55         iv = encrypted_message[:AES.block_size]
56         cipher = AES.new(key, mode, iv)
57         if mode in [AES.MODE_CFB, AES.MODE_OFB]:
58             decrypted_message = cipher.decrypt(encrypted_message[AES.block_size:])
59         else:
60             decrypted_message = unpad(cipher.decrypt(encrypted_message[AES.block_size:]), AES.block_size)
61
62     elif mode in [AES.MODE_CCM, AES.MODE_GCM, AES.MODE_OCB, AES.MODE_CTR]:
63         nonce = encrypted_message[:12] # 12-byte nonce
64         cipher = AES.new(key, mode, nonce=nonce)
65         if mode in [AES.MODE_CCM, AES.MODE_GCM, AES.MODE_OCB]:
66             tag = encrypted_message[-16:]
67             encrypted_message = encrypted_message[12:-16]
68             decrypted_message = cipher.decrypt_and_verify(encrypted_message, tag)
69         else:
70             decrypted_message = cipher.decrypt(encrypted_message[12:])
71
72     else:
73         cipher = AES.new(key, mode)
74         if mode in [AES.MODE_ECB]:
75             decrypted_message = unpad(cipher.decrypt(encrypted_message), AES.block_size)
76         else:
77             decrypted_message = cipher.decrypt(encrypted_message)
78
79     return decrypted_message.decode()
80

```

Example Usage

```
1 # Dictionary mapping mode indices to AES encryption modes
2 available_modes = {
3     1: AES.MODE_ECB, # Electronic Codebook Mode
4     2: AES.MODE_CBC, # Cipher Block Chaining Mode
5     3: AES.MODE_CFB, # Cipher Feedback Mode
6     5: AES.MODE_OFB, # Output Feedback Mode
7     6: AES.MODE_CTR, # Counter Mode
8     8: AES.MODE_CCM, # Counter with CBC-MAC Mode (Provides authentication)
9     11: AES.MODE_GCM, # Galois/Counter Mode (Authenticated encryption)
10    12: AES.MODE_OCB # Offset Codebook Mode (Authenticated encryption)
11 }
12
13 available_modes_to_string = {
14     1: "AES.MODE_ECB", # Electronic Codebook Mode
15     2: "AES.MODE_CBC", # Cipher Block Chaining Mode
16     3: "AES.MODE_CFB", # Cipher Feedback Mode
17     5: "AES.MODE_OFB", # Output Feedback Mode
18     6: "AES.MODE_CTR", # Counter Mode
19     8: "AES.MODE_CCM", # Counter with CBC-MAC Mode (Provides authentication)
20     11: "AES.MODE_GCM", # Galois/Counter Mode (Authenticated encryption)
21     12: "AES.MODE_OCB" # Offset Codebook Mode (Authenticated encryption)
22 }
23
24 # Loop through each available encryption mode
25 for m in available_modes:
26     print(f"Mode {available_modes_to_string[m]}")
27     passphrase = "Max Verstappen" # Secret key used for encryption
28     message = "I'm moving up and down, side to side like a roller coaster" # Message to be encrypted
29     key_length = 256 # AES key length in bits (valid options: 128, 192, 256)
30
31     mode_idx = m # Current mode index
32     mode = available_modes[mode_idx] # Retrieve AES mode from dictionary
33
34     # Encrypt the message using the specified AES mode
35     encrypted_message = encrypt(message, passphrase, key_length, mode)
36     print("Encrypted message:", encrypted_message)
37
38     # Decrypt the message to verify correctness
39     decrypted_message = decrypt(encrypted_message, passphrase, key_length, mode)
40     print("Decrypted message:", decrypted_message, '\n')
41
```

 Mode AES.MODE_ECB
Encrypted message: EbbGhFgG0uP/4aWsx+WLGDt8NesDCbAgP0f00qXwEpX6TmEjPBX/tr5jRlie87bVCp4uTK1gprI6p9S/20QipA==
Decrypted message: I'm moving up and down, side to side like a roller coaster

Mode AES.MODE_CBC
Encrypted message: vHghQW5wW0uH0n9VIErUbhQd1yH39K5gtsWJTjZt4mBAvN8NVC4rjRhwWNMIeqc8ZDnI0KplSRGArSjhw5rlbYdswGFmRaVstKIhnm.
Decrypted message: I'm moving up and down, side to side like a roller coaster

Mode AES.MODE_CFB
Encrypted message: C0ZnNEQdZwn4whG2ZC9KB04+kgx8kEzN5/EpJucP8PA1xQrIANwM041fgaPKmfN//0Goo+oR4A37k2cY0lgSU/G4zQL9YBWo2eA=
Decrypted message: I'm moving up and down, side to side like a roller coaster

Mode AES.MODE_OFB
Encrypted message: F95g2evWscKLIsy+kwUiFL78x8gHFvt5BT0UjALPGM2j+07HAKb0kZo96Bd320vPwH2xEHtfW1W4CD9cNpRKpxgNdCd7IFcY++o=
Decrypted message: I'm moving up and down, side to side like a roller coaster

Mode AES.MODE_CTR
Encrypted message: 3RgxfvtlVWn9xSPRMkDJbqXxCyKAoxk4Tl8BxyjVDzIhDUzDwMaWmmQwNLzWljn0R37y58apvjqq4QpoRPIe5t5FvNrD1w==
Decrypted message: I'm moving up and down, side to side like a roller coaster

Mode AES.MODE_CCM
Encrypted message: dUT7+wLpQYFyC898jlQcNwL4T9GLrTW8DVHz4AdmPUWsnLGsj7qU64yvVB2XanNRVMJhBb7aXIc401VJnR3D8y18IPiJP0okLWFg1l
Decrypted message: I'm moving up and down, side to side like a roller coaster

Mode AES.MODE_GCM
Encrypted message: 5eDfUI30evFWRin0WsDqflu2SMY0QwZuvlfQLxKs9rEMtjhibEJH7ryjZNZGzyODQUKtmX9sfVztEkdGIlgzRR5P6iztYsc4DwAtP.
Decrypted message: I'm moving up and down, side to side like a roller coaster

Mode AES.MODE_OCB
Encrypted message: 6x/piraf5PNYEUzsTROcpdxN/jsSiqSnrLalxBvvzXrBNCYRjJdViSfCUQTJ5yz08DwS2RSXodh0JG1xtQhuYLx3EInJZ8M+NMfUI
Decrypted message: I'm moving up and down, side to side like a roller coaster

✓ Rivest, Shamir, Adleman (RSA)

Available functions:

- `generate_rsa_keys(bits)` → `private_key`, `public_key`, `rsa_components`
- `save_rsa_keys(private_key, public_key, private_key_filename, public_key_filename)` → `None`
- `import_rsa_key(key_filename)` → RSA key object
- `read_rsa_key(key_string)` → RSA key object
- `encrypt_rsa(message, public_key)` → `encrypted_message`
- `decrypt_rsa(encrypted_message, private_key)` → `message`
- `sign_message_pss(message, private_key)` → `signature`
- `verify_signature(message, signature, public_key)` → `True/False`

Supported RSA key sizes (in bits):

- 1024
- 2048
- 3072
- 4096

```
1 # RSA docs: https://pycryptodome.readthedocs.io/en/latest/src/public_key/rsa.html
2 # RSASSA-PSS docs: https://pycryptodome.readthedocs.io/en/latest/src/signature/pkcs1_pss.html
3
4 def generate_rsa_keys(bits=2048):
5     """Generates a pair of RSA keys (public and private).
6
7     Args:
8         bits: The number of bits for the RSA key.
9
10    Returns:
11        A tuple containing the private key and public key.
12    """
13    new_key = RSA.generate(bits)
14    private_key = new_key.export_key(format='PEM')
15    public_key = new_key.publickey().export_key(format='PEM')
16    return private_key, public_key, {'n': new_key.n, 'e': new_key.e, 'd': new_key.d, 'p': new_key.p, 'q':
17
18
19 def save_rsa_keys(private_key, public_key, private_key_filename="private_rsa.pem", public_key_filename='
20     """Saves the RSA keys to files.
21
22    Args:
23        private_key: The private key.
24        public_key: The public key.
25        private_key_filename: The filename for the private key file.
26        public_key_filename: The filename for the public key file.
27    """
28    with open(private_key_filename, 'wb') as f:
29        f.write(private_key)
30    with open(public_key_filename, 'wb') as f:
31        f.write(public_key)
32
33
34 def import_rsa_key(key_filename):
35     """Imports an RSA key from a file.
36
37    Args:
38        key_filename: The filename of the key file.
39
40    Returns:
41        The RSA key object.
42    """
43    with open(key_filename, 'rb') as f:
44        key = RSA.import_key(f.read())
45    return key
46
47 def read_rsa_key(rsa_key):
48     """Reads an RSA key from a string.
49
50    Args:
51        rsa_key: The RSA key as a string.
52
```

```

53 Returns:
54     The RSA key object.
55     """
56     return RSA.import_key(rsa_key)
57
58
59 def encrypt_rsa(message, public_key):
60     """Encrypts a message using RSA.
61
62     Args:
63         message: The message to encrypt.
64         public_key: The RSA public key.
65
66     Returns:
67         The encrypted message.
68     """
69     cipher_rsa = PKCS1_OAEP.new(public_key)
70     encrypted_message = cipher_rsa.encrypt(message.encode())
71     return binascii.hexlify(encrypted_message).decode('ascii')
72
73
74 def decrypt_rsa(encrypted_message, private_key):
75     """Decrypts a message using RSA.
76
77     Args:
78         encrypted_message: The encrypted message.
79         private_key: The RSA private key.
80
81     Returns:
82         The decrypted message.
83     """
84     cipher_rsa = PKCS1_OAEP.new(private_key)
85     decrypted_message = cipher_rsa.decrypt(binascii.unhexlify(encrypted_message))
86     return decrypted_message.decode()
87
88 def sign_message_pss(message, private_key):
89     """Signs a message using PKCS#1 PSS (RSASSA-PSS).
90
91     Args:
92         message: The message to sign.
93         private_key: The RSA private key.
94
95     Returns:
96         The signature.
97     """
98     hash_obj = SHA256.new(message.encode())
99     signer = pss.new(private_key)
100    signature = signer.sign(hash_obj)
101    return binascii.hexlify(signature).decode('ascii')
102
103
104 def verify_signature_pss(message, signature, public_key):
105     """Verifies an PKCS#1 PSS (RSASSA-PSS) signature.
106
107     Args:
108         message: The message that was signed.
109         signature: The signature to verify.
110         public_key: The RSA public key.
111
112     Returns:
113         True if the signature is valid, False otherwise.
114     """
115     hash_obj = SHA256.new(message.encode())
116     verifier = pss.new(public_key)
117     try:
118         signature = binascii.unhexlify(signature)
119         verifier.verify(hash_obj, signature)
120         return True
121     except ValueError:
122         return False

```

Example Usage

```

1 # List of possible RSA key lengths in bits
2 key_lengths = [1024, 2048, 3072, 4096]
3
4 # Index to select the desired key length (change kl_idx to use a different key length)
5 kl_idx = 0
6
7 # Message to be encrypted and signed
8 message = "Must be the water"
9
10 # Generate RSA keys (private and public) with the selected key length
11 private_key, public_key, rsa_components = generate_rsa_keys(bits=key_lengths[kl_idx])
12
13 # Save the generated RSA keys to files for later use
14 save_rsa_keys(private_key, public_key)
15
16 # Import (load) the saved public key from file
17 public_key_imported = import_rsa_key("public_rsa.pem")
18
19 # Import (load) the saved private key from file
20 private_key_imported = import_rsa_key("private_rsa.pem")
21
22 # Print the private and public keys as decoded strings
23 print(f"Private key: {private_key.decode()} \n")
24 print(f"Public key: {public_key.decode()} \n")
25
26 # Encrypt the message using the imported public key
27 encrypted_message = encrypt_rsa(message, public_key_imported)
28
29 # Decrypt the encrypted message using the imported private key
30 decrypted_message = decrypt_rsa(encrypted_message, private_key_imported)
31
32 # Display original, encrypted, and decrypted messages
33 print("Original message:", message)
34 print("Encrypted message:", encrypted_message)
35 print("Decrypted message:", decrypted_message, '\n')
36
37 # Generate a digital signature for the message using PSS (Probabilistic Signature Scheme)
38 signature = sign_message_pss(message, private_key_imported)
39 print(f"Signature: {signature}")
40
41 # Verify the signature using the public key
42 is_valid = verify_signature_pss(message, signature, public_key_imported)
43 print(f"Signature is valid: {is_valid}")
44

```

```

🔓 Private key: -----BEGIN RSA PRIVATE KEY-----
MIICXAIBAAKBgQDfP1/tD01ZVqz/wL8Fh+FOStLInzK4RzJE+DtZkT2HFb00Jpok
RT2Pb3Kt89QWHAQn4CdiR9gP4HqeDp1F0pBQz6dkf9+5K0dpTLsfw4SZ3f7Bg4V
dMk2Fs20AY30Ywpi0jWQvA1P7l8/7ptviR3wnxFc49xbdzX7hWsu1WZ7KQIDAQAB
AoGAEQvwd0BJ4XERF7PQ1nGnNkDXbWT7yUsG5EjH0U8I8Bwf+0o0Q1r0DmnBcZjr
XQNZjoI4Z3T+sA7hA5JjKpH3iig7Wrg6QIesMCDzrFP9e7UrqQBwn3jPUMz66sDG
kNeiR5fJ2RDbEiWB92rKZzeMPGHCgFPnGLT1Z62Z0NSb8iECQQDwdeHtuoBkYjWu
JU0Iv15D9rkbsT79DFLhpjbU1NvLcSaGoVXEuy40Ud0D7ZxBmSdP5j35uHI3bfJ4
aUcWpRFJAKEA63P7XM/D5hNx58BH5i/0mH4GEb5Uc01AMM92sS4zXnqux8y2dl6f
n9IT50H2vXBNn4000hBnJotNRbvLwOn64QJBAMg3RbWraDJPfc4XBNQk5dyNekNg
Nm4IEiW1JneH1TPTihidra0D+7If04sRrT3RrxRN4rrp7YX70Z7AuMh0AECQDdK
jzamW5yF6HPBxaCajWt1/Mrh5TZ2r+pGbdSbvHJtY0nuetK10drwHFhsNyMZYkEZ
TrcdvvtgW83IIv25kECQE0A5PeVVX+mT5HYjovdfIooGn514rv6+CqjUtYodhQQ
Nfxv8xxatWmAVnlsR8pA26ZsiYpgSaTvcCjugNz6aIA=
-----END RSA PRIVATE KEY-----

Public key: -----BEGIN PUBLIC KEY-----
MIGfMA0GCsQgIb3DQEBQUAA4GNADCBiQKBgQDfP1/tD01ZVqz/wL8Fh+FOStLI
nzK4RzJE+DtZkT2HFb00JpokRT2Pb3Kt89QWHAQn4CdiR9gP4HqeDp1F0pBQz6d
kf9+5K0dpTLsfw4SZ3f7Bg4VdMk2Fs20AY30Ywpi0jWQvA1P7l8/7ptviR3wnxFc
49xbdzX7hWsu1WZ7KQIDAQAB
-----END PUBLIC KEY-----

Original message: Must be the water
Encrypted message: 977cfec9bc996b8f0e1b89cbea874b5d17e8c6d5a1047c5a9890d7049b5a8885b0380680b330a1d201173230d7d2365f19a9f
Decrypted message: Must be the water

Signature: 8855494e8e20d301656ce0021179fcee88963a51918049718c4188a3dbdc2de7dc8259c05f98b45d41ae261be44797f097cc049ba7f3al
Signature is valid: True

```

↩ Elliptic Curves Digital Signature Algorithm (ECDSA)

Available functions:

- `generate_ecc_keys(curve)` → `private_key`, `public_key`
- `import_ecc_key(key_filename)` → ECC key object
- `read_ecc_key(key_string)` → ECC key object
- `export_ecc_key(key)` → `None`
- `sign_message_ecc(message, private_key)` → `signature`
- `verify_signature_ecc(message, signature, public_key)` → `True/False`

Supported curves:

- NIST P-192
- NIST P-224
- NIST P-256
- NIST P-384
- NIST P-521
- Ed25519
- Ed448

```
1 # ECC docs: https://pycryptodome.readthedocs.io/en/latest/src/public_key/ecc.html
2 # ECDSA docs: https://pycryptodome.readthedocs.io/en/latest/src/signature/dsa.html
3
4 def generate_ecc_keys(curve='NIST P-192'):
5     """Generates a pair of ECC keys (private and public).
6
7     Returns:
8         A tuple containing the private key and public key.
9     """
10    key = ECC.generate(curve=curve)
11    private_key = key.export_key(format='PEM')
12    public_key = key.public_key().export_key(format='PEM')
13    return private_key, public_key
14
15
16 def import_ecc_key(key_filename):
17     """Imports an ECC key from a file.
18
19     Args:
20         key_filename: The filename of the key file.
21
22     Returns:
23         The ECC key object.
24     """
25    with open(key_filename, 'rb') as f:
26        key = ECC.import_key(f.read())
27    return key
28
29
30 def read_ecc_key(key):
31     """Reads an ECC key from a string.
32
33     Args:
34         key: The ECC key as a string.
35
36     Returns:
37         The ECC key object.
38     """
39    return ECC.import_key(key)
40
41 def export_ecc_key(key):
42     """Exports an ECC key to PEM format.
43
44     Args:
45         key: The ECC key object.
46
47     Returns:
48         The ECC key in PEM format.
49     """
50    return key.export_key(format='PEM')
51
52
53 def sign_message_ecc(message, private_key):
54     """Signs a message using ECDSA.
```

```

55
56 Args:
57     message: The message to sign.
58     private_key: The ECC private key.
59
60 Returns:
61     The signature.
62 """
63 hash_obj = SHA256.new(message.encode())
64 signer = DSS.new(private_key, 'fips-186-3')
65 signature = signer.sign(hash_obj)
66 return signature
67
68 def verify_signature_ecc(message, signature, public_key):
69     """Verifies an ECDSA signature.
70
71     Args:
72         message: The message that was signed.
73         signature: The signature to verify.
74         public_key: The ECC public key.
75
76     Returns:
77         True if the signature is valid, False otherwise.
78     """
79     hash_obj = SHA256.new(message.encode())
80     verifier = DSS.new(public_key, 'fips-186-3')
81     try:
82         verifier.verify(hash_obj, signature)
83         return True
84     except ValueError:
85         return False
86
87

```

▼ Example Usage

```

1 # List of available elliptic curves for ECC key generation
2 available_curves = ['NIST P-192', 'NIST P-224', 'NIST P-256',
3                     'NIST P-384', 'NIST P-521', 'Ed25519', 'Ed448']
4
5 # Index to select the desired elliptic curve
6 idx_curve = 0 # Change this index to use a different curve
7
8 # Generate an ECC key pair using the selected curve
9 private_key, public_key = generate_ecc_keys(available_curves[idx_curve])
10
11 # Print the generated private and public keys
12 print(f"Private key: {private_key}\n")
13 print(f"Public key: {public_key}\n")
14
15 # Message to be signed
16 message = "Valterri, it's James..."
17
18 # Sign the message using the private key (imported for use)
19 signature = sign_message_ecc(message, read_ecc_key(private_key))
20 print(f"Signature: {signature} \n")
21
22 # Verify the signature using the public key (imported for use)
23 is_valid = verify_signature_ecc(message, signature, read_ecc_key(public_key))
24 print(f"Signature is valid: {is_valid} \n")
25

```

```

🔑 Private key: -----BEGIN PRIVATE KEY-----
MG8CAQAwEYHKoZIzj0CAQYIKoZIzj0DAQEEVTBTAgEBBBheYaSRp1EWGCRPyGsT
X24pL7URa30rz5GhNAMYAAQHuf+M+vpp4zj4u6s/BakbSyY3Dbebb0gPIZs75Vvk6
Kx3F+sIxGQRJAXwd0GFo8Sk=
-----END PRIVATE KEY-----

Public key: -----BEGIN PUBLIC KEY-----
MEkwEwYHKoZIzj0CAQYIKoZIzj0DAQEDMgAEB7n/jPr6aeM4+LurPwWim0smNw23
m2zoDyGb0+VZOisdxfrcMRkESQMCHThhaPEp
-----END PUBLIC KEY-----

Signature: b'\x8bc\xeb\x1d\x80\x11C\x81\x870\x0f\xca\x1d\xfe\xc6}\xf1e\x00\xa3\xbeY\xdf\x14\xcd\xe9\xa09\xf3\xed\xc9\xe6'

```

Signature is valid: True

✓ Assignment

✓ Symmetric Encryption

In this assignment, students will explore AES encryption by performing the following tasks:

1. Encrypt a message: use AES with a chosen mode of operation and key length to encrypt a given message using a pre-shared secret key.
2.  Publish to JSONbin: upload the encrypted message to JSONbin
3. Retrieve and Decrypt: fetch an encrypted message previously published by your friend on JSONbin and decrypt it using the correct passphrase, key length, and mode of operation.
4. Analyze AES-GCM Behavior: encrypt and decrypt the same message multiple times using AES-GCM-128 and explain why the ciphertext varies.
5. Test Mode & Key Length Variations: encrypt a message using six different combinations of AES modes and key lengths, then analyze how these factors affect ciphertext length.
6. Discussion: briefly explain the impact of mode of operation selection, key length, IV/nonce, and authentication tags on ciphertext size and security.

```
1 # Pycryptodome AES docs: https://www.pycryptodome.org/src/cipher/aes
2
3 available_modes = {
4     1: AES.MODE_ECB,
5     2: AES.MODE_CBC,
6     3: AES.MODE_CFB,
7     5: AES.MODE_OFB,
8     6: AES.MODE_CTR,
9     8: AES.MODE_CCM,
10    11: AES.MODE_GCM,
11    12: AES.MODE_OCB,
12 }
13
14 available_AES_key_lengths = [128, 192, 256] # key length in bits
```

```
1 pre_shared_secret_password = "T0-D0" # Add here the shared secret password you exchanged with you
2 message_to_be_exchanged = "T0-D0" # Add here the message you want to exchange
3
4 # T0-D0: Encrypt the message using AES, a mode of operation and key length of your choosing, and the pre-
5
6
7 # T0-D0: Publish the encrypted message in JSONbin
8
9
10 # T0-D0: Retrieve the encrypted message from JSONbin (your friend message! Not the message you just publi
11
12
13 # T0-D0: Decrypt the message from your friend
14
15
```

```
1 # T0-D0: Try to encrypt and decrypt the same message multiple times using AES-GCM-128. Why is the ciphert
2
3
4 # T0-D0: Try different combinations of modes of operation and key lengths (6 combinations in total is sui
5
6
7 # T0-D0: Briefly discuss how different combination of mode of operation and key length affect ciphertext
8
```

✓ Asymmetric Encryption

In this assignment, students will work with RSA encryption by performing the following tasks:

1. Generate RSA Key Pairs: create a new pair of RSA private/public keys with a selected key length and save them.

2. Print Keys: display the generated private and public keys in readable format.
3. Import RSA Keys: load previously saved RSA keys from files for encryption and decryption.
4. Encrypt a Message: request a friend's public key, use it to encrypt a confidential message, and publish the ciphertext to JSONbin.
5. Retrieve and Decrypt: fetch an encrypted message previously published by your friend on JSONbin and decrypt it using the correct private key.
6. Experiment with Key Lengths: generate RSA keys with different key lengths (1024, 2048, 3072, 4096 bits) and analyze the key components (n, e, d, p, q).
7. Discussion: briefly compare how key length impacts security, performance, and ciphertext size.

```
1 rsa_key_lengths = [1024, 2048, 3072, 4096] # Available key lengths in bit
2
3 # T0-D0: Generate and save a new pair of RSA private/public keys
4
5
6 # T0-D0: Print the just generated private and public keys
7
8
```

```
1 # T0-D0: Import your private and public RSA keys
2
3
4 # T0-D0: You want to send a confidential message to your friend... Ask them to share their public key with
5
6
7 # T0-D0: Encrypt the message using your friend public key and publish the encrypted message in JSONbin
8
9
10 # T0-D0: Retrieve the encrypted message from JSONbin (your friend message! Not the message you just published)
11
12
13 # T0-D0: Decrypt the message from your friend using your own private key
14
15
16 # Display original, encrypted, and decrypted messages
17
```

```
1 # T0-D0: Try different RSA key lengths and inspect the RSA parameters (i.e., n, e, d, p, q)
2
3
```

✓ Digital Signatures

In this assignment, students will explore digital signatures and message authentication using RSASSA-PSS and ECDSA. They will complete the following tasks:

1. Sign and Encrypt a Message: encrypt a message using RSA, generate a digital signature, and publish both the message and the signature to JSONbin (use the same JSONbin).
2. Retrieve and Verify: fetch an encrypted message and its signature from JSONbin, verify its authenticity using the sender's public key, and decrypt it.
3. Key Usage Discussion: explain which keys were used for encryption/decryption versus signature generation/validation.
4. Signature Experiments: generate and compare signatures using different key types (e.g., RSA 1024-bit and 4096-bit, ECDSA NIST P-192, P-224, and P-521).
5. Key & Signature Analysis: briefly discuss how key type and length affect signature size, security, and performance in RSA vs. ECC.

```
1 # Now you want to be sure that the message you received is from your friend
2
3 # List of available elliptic curves for ECC key generation
4 available_curves = ['NIST P-192', 'NIST P-224', 'NIST P-256',
5                     'NIST P-384', 'NIST P-521', 'Ed25519', 'Ed448']
6
7 rsa_key_lengths = [1024, 2048, 3072, 4096] # Available key length in bit
8
9
10 # T0-D0: Follow the same procedure as for asymmetric encryption, but this time publish the signature along
```

```
11 # You can either use PKCS#1 PSS (RSA) or ECDSA to sign the message.
12
13
14 # T0-D0: Discuss which keys you used for message encryption/decryption and which ones for signature generation.
15
16

1 # T0-D0: Try to generate signatures with different combination of (RSA, RSA_KEY_LENGTH) and (ECDSA, CURVE).
2
3
4 # T0-D0: Briefly discuss how key and signature sizes change between RSA and EC.
5
6
```

Final discussion and report

The report should be divided in three sections addressing the following topics:

- **Symmetric encryption**
 - Describe the various steps you followed to encrypt, upload, download, and decrypt the secret messages.
 - Discuss which information you needed to share with your friend to enable the exchange of the encrypted messages.
 - Briefly discuss how different modes of operation and key lengths affect ciphertext length and security.
- **Asymmetric encryption**
 - Describe the various steps you followed to encrypt, upload, download, and decrypt the secret messages.
 - Discuss which information you needed to share with your friend to enable the exchange of the encrypted messages.
 - Regarding the previous point, compare with what was required when using symmetric encryption. What are the pros and cons of each cryptography scheme?
 - How does the RSA key length relate to the length (in bits) of the RSA components (i.e., n , p , q , e , d)?
- **Digital signatures**
 - Describe the various steps you followed to encrypt, sign, upload, download, decrypt, and verify the secret signed messages. In particular, highlight which private and public key were used for each operation.
 - Briefly discuss and compare (qualitatively) how PKCS#1 PSS and ECDSA differ in terms of: private and public key length, signature size, security, and performances.